Ex 2

Aim To perform Exploratory Data Analysis (EDA) and data cleaning on the OnlineRetail dataset by applying standard data science frameworks (CRISP-DM, SEMMA, and KDD).

Procedure

- **Framework Implementation**: Applied KDD/SEMMA/CRISP-DM stages for data selection, preprocessing, transformation, and evaluation.
- **Data Cleaning**: Removed null values and duplicate rows, normalized column text, and checked statistics using Pandas.
- **Data Storage**: Exported the processed dataset into CSV, JSON, and an SQLite database for structured querying.

Program :

```
df_retail = pd.read_csv('/content/OnlineRetail.csv', encoding='ISO-8859-1')
display(df_retail.head())
```

22] Python

```
print(df_retail.info())
print(df_retail.shape)
print(df_retail.columns)

# Clean column names (remove leading/trailing spaces)
df_retail.columns = df_retail.columns.str.strip()

# Convert object columns to lowercase (if applicable, though 'InvoiceNo' and 'St
for col in df_retail.select_dtypes(include='object').columns:
    df_retail[col] = df_retail[col].astype(str).str.lower()

display(df_retail.head())
```

23] Python

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
 #   Column       Non-Null Count   Dtype
---  ------       --------------   -----
 0   InvoiceNo    541909 non-null  object
 1   StockCode    541909 non-null  object
 2   Description  540455 non-null  object
 3   Quantity     541909 non-null  int64
 4   InvoiceDate  541909 non-null  object
 5   UnitPrice    541909 non-null  float64
 6   CustomerID   406829 non-null  float64
 7   Country      541909 non-null  object
dtypes: float64(2), int64(1), object(5)
memory usage: 33.1+ MB
```

```
print(df_retail.isnull().sum())

# Drop rows with any missing values
df_retail = df_retail.dropna()

print(f"Shape after dropping nulls: {df_retail.shape}")
```

[24] Python

```
InvoiceNo          0
StockCode          0
Description        0
Quantity           0
InvoiceDate        0
UnitPrice          0
CustomerID     135080
Country            0
dtype: int64
Shape after dropping nulls: (406829, 8)
```

Handle Duplicate Rows

```
print(f"Number of duplicated rows: {df_retail.duplicated().sum()}")

# Drop duplicate rows
df_retail = df_retail.drop_duplicates()

print(f"Shape after dropping duplicates: {df_retail.shape}")
```

[25] Python

```
Number of duplicated rows: 5225
Shape after dropping duplicates: (401604, 8)
```

Descriptive Statistics

```
print("\nDescriptive statistics for numerical columns:")
display(df_retail.describe())

print("\nDescriptive statistics for object columns:")
display(df_retail.describe(include='object'))
```

[26] Python

```
Descriptive statistics for numerical columns:
```

```
Descriptive statistics for object columns:
```

Save Cleaned Data

```
# Save to CSV
df_retail.to_csv("cleaned_online_retail.csv", index=False)

# Save to JSON
df_retail.to_json("cleaned_online_retail.json", orient="records", indent=4)
print("Cleaned data saved to 'cleaned_online_retail.csv' and 'cleaned_online_ret
```

[28] Python

```
Cleaned data saved to 'cleaned_online_retail.csv' and 'cleaned_online_retail.json'.
```

Load Data into SQLite Database and Perform Queries

```
# Establish SQLite connection and load data
connection_retail = sqlite3.connect("online_retail.db")
df_retail.to_sql("retail_transactions", connection_retail, if_exists="replace",
print(f"Successfully loaded {df_retail.shape[0]} records into 'retail_transactio
```

[29] Python

```
Successfully loaded 401604 records into 'retail_transactions' table.
```

```
print("\nFirst 5 rows from the database:")
display(pd.read_sql("SELECT * FROM retail_transactions LIMIT 5;", connection_ret
```

[30] Python

```
Minimum UnitPrice:
```

```
print("\nTotal sum of Quantity:")
display(pd.read_sql("SELECT SUM(Quantity) AS total_quantity FROM retail_transact
```

[46] Python

```
Total sum of Quantity:
```

```
print("\nCountries with more than 1000 transactions:")
display(pd.read_sql("SELECT Country, COUNT(*) AS total FROM retail_transactions
```

[48] Python

```
Countries with more than 1000 transactions:
```

```
print("\nFirst 10 records from the database:")
display(pd.read_sql("SELECT * FROM retail_transactions LIMIT 10;", connection_re
```

[47] Python

```
First 10 records from the database:
```

Generate + Code + Markdown

```
# Close the database connection
connection_retail.close()
print("SQLite connection closed.")
```

[35] Python

```
SQLite connection closed.
```

Result Thus the EDA and data cleaning using data science frameworks like CRISP-DM, SEMMA, KDD is implemented successfully.